

IRD MAX

WRO Future Engineers

TEAM #5865 / RIYADH, SAUDI ARABIA



مدارس ابن رشد العالمية
IBN ROSHD INTERNATIONAL SCHOOLS

Abdulaziz Nasser Al-Mindil

Mohamed Aldawood

Coach: Engineer Mohammed Emam

ENGINEERING DOCUMENTATION REPORT

Table of Contents

A compact guide to the robot, its development process and the evidence in the repository.

01	Team & Robot Overview	3
02	System Architecture	4
03	Design Ideas & Strategy	5
04	CAD & Mechanical Layout	6
05	Hardware	7
06	Schematics	8
07	Power & Pin Map	9
08	Software & Code	10
09	Calibration & Setup	11
10	Testing & Results	12
11	Problems & Solutions	13
12	Conclusion & Repository Guide	15

SOURCE SCOPE

Content is summarized from the repository's README, CAD, hardware, schematic, software, testing and journal pages.

Team & Robot Overview

PROJECT SUMMARY

IRD MAX is a self-driving car created for WRO Future Engineers. The repository documents the current vehicle, challenge code, mechanical design, wiring, calibration tools, tests and development journal.



CURRENT VEHICLE

TEAM

NAME	IRD MAX
NUMBER	#5865
CITY	Riyadh
COUNTRY	Saudi Arabia
COMPETITION	WRO Future Engineers

PEOPLE

Abdulaziz Nasser Al-Mindil

Mohamed Aldawood

Coach: Engineer Mohammed Emam

ROBOT AT A GLANCE

COMPUTE

Original NVIDIA Jetson Nano

Processes the camera, identifies colours and chooses driving actions.

CONTROL

Arduino Uno R3

Reads sensors and produces the drive-motor and steering-servo signals.

VISION

IMX477 camera

Detects blue/orange floor lines and red/green traffic pillars.

MOTION

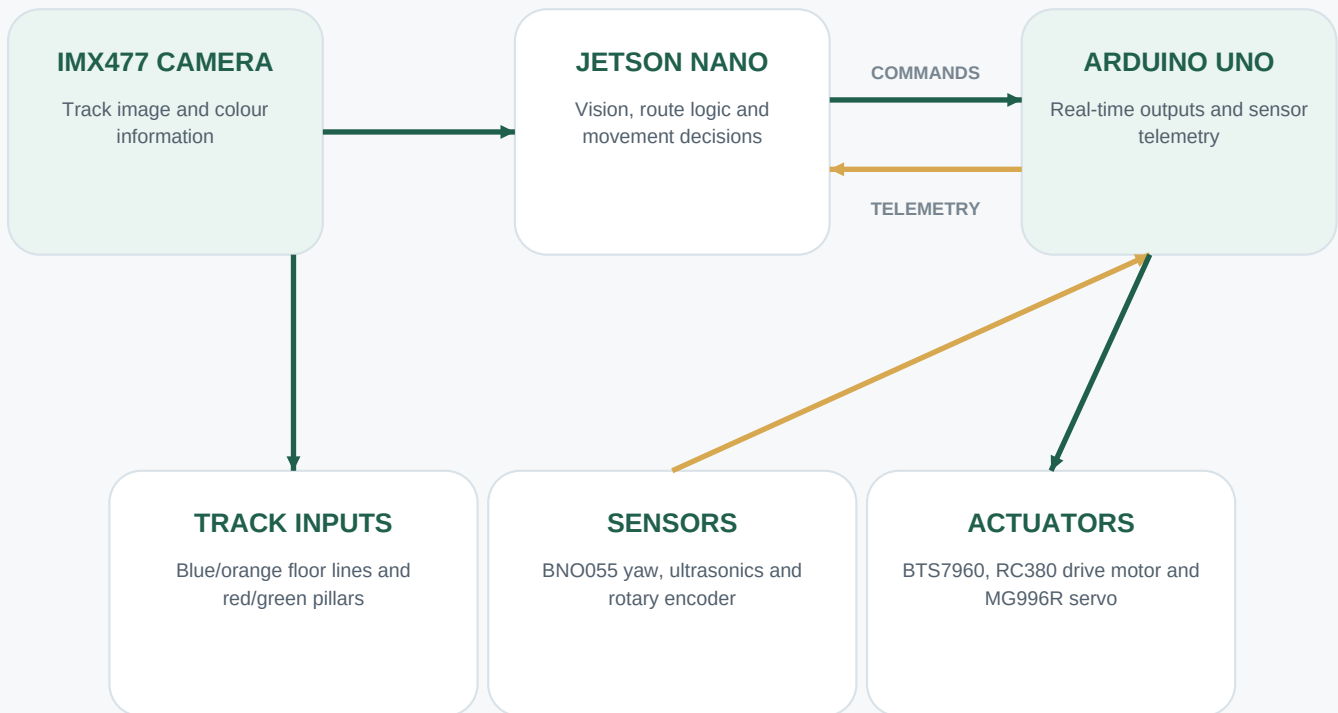
Rear drive + front steering

RC380 motor drives the rear wheels; MG996R turns the front wheels.

System Architecture

DATA AND CONTROL

The design separates high-level perception and decisions from low-level motor, steering and sensor handling. USB serial at 115200 baud carries commands and telemetry between the two boards.



CONTROL LOOP

- 1 SEE** Camera frames and sensor readings describe the current track state.
- 2 DECIDE** The Jetson chooses straight driving, a corner turn, pillar avoidance or parking.
- 3 ACT** The Uno applies motor PWM, steering and absolute-yaw turn commands.
- 4 CORRECT** Fresh yaw and distance telemetry are used to update the next command.

Design Ideas & Strategy

FROM RULES TO BEHAVIOUR

The route logic combines floor-line detection, heading control, ultrasonic distances and colour-based pillar decisions. Left and right are always measured from the car's direction of travel.

OBSTACLE ROUTE



Start at the thick red line on the right, travel upward and follow the marked route counterclockwise.



Red pillar

Pass on the pillar's right side. After clearing it, return toward the straight target heading.



Green pillar

Pass on the pillar's left side. When the manoeuvre ends, re-enter the normal heading-control loop.

DESIGN IDEAS

Line-aware corners

Blue and orange floor lines identify intersections and corner direction.

Straight-heading hold

A target yaw is held on each straight using fresh, smoothed corrections.

Sensor confirmation

Ultrasonic distances and encoder movement confirm position when vision alone is not enough.

Separated challenges

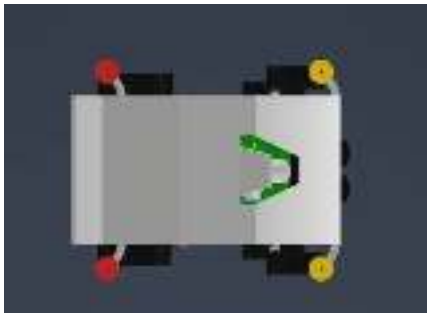
Open and Obstacle programs use matched Arduino controllers because their telemetry and movement needs differ.

CAD & Mechanical Layout

MECHANICAL PLANNING

The CAD model was used to position the chassis, drivetrain, steering, electronics, camera, sensors and upper cover before assembly. The screenshots document layout intent rather than dimensioned manufacturing drawings.

TOP



FRONT



LEFT



RIGHT



LAYOUT REASONS

- Rear drive and front steering keep the mechanism straightforward and compact.
- Heavy parts stay low and near the centre to improve stability while turning.
- The camera sits high at the front, while sensors face outward without body-panel blockage.
- Clearance around the steering linkage allows the wheels to turn without contacting the cover.

Hardware

INSTALLED PLATFORM

The current car uses the original Jetson Nano and an Arduino Uno R3. The major components are chosen to separate vision processing, real-time control, sensing, steering and rear-wheel drive.



JETSON NANO



RC380 MOTOR



STEP-DOWN REGULATOR

COMPONENT LIST

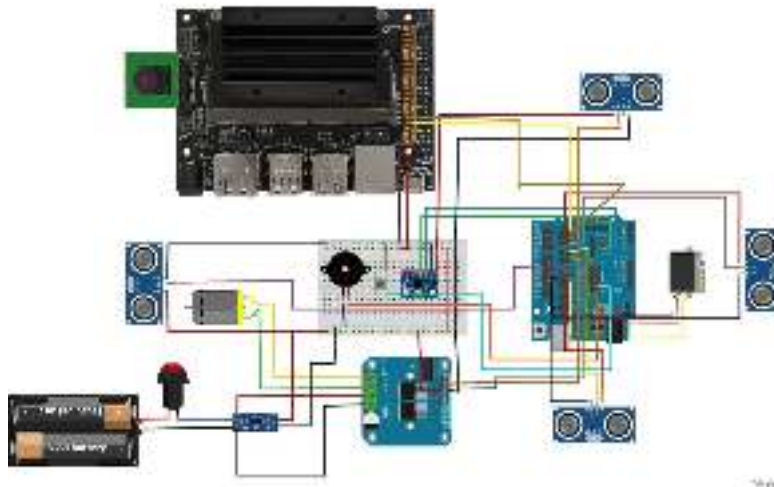
COMPONENT	QTY	ROLE
NVIDIA Jetson Nano	1	Vision and decision computer
Arduino Uno R3 + shield	1	Motor, steering and sensor control
IMX477 camera	1	Track and pillar vision
DFRobot ultrasonic sensors	4	Front, left, right and rear distance
DFRobot BNO055 IMU	1	Yaw / heading
RC380 brushed motor	1	Rear-wheel drive; marked 7.2 V
MG996R steering servo	1	Front steering
Rotary encoder	1	Travel-distance estimate
HW-039 / BTS7960 module	1	High-current motor driver
Buzzer, start button, holder	1 each	User feedback, start and battery support

Schematics

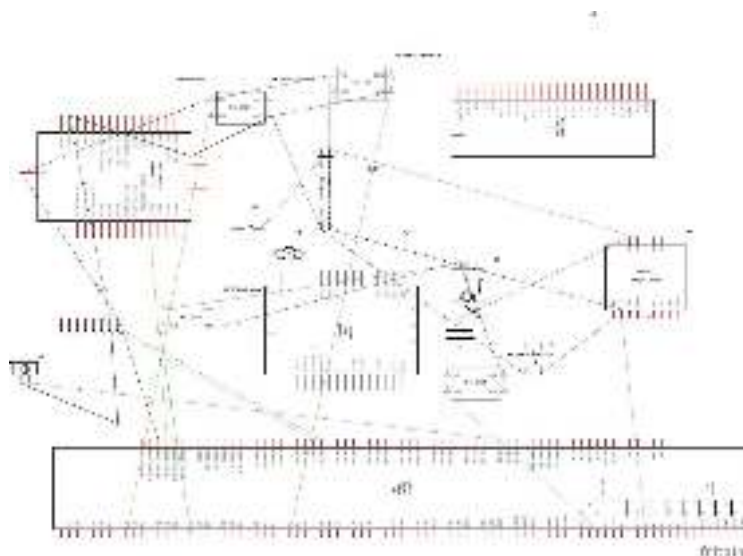
CONNECTION DOCUMENTATION

The repository provides two complementary wiring views. The circuit diagram focuses on component connections, while the machine schematic expresses the same system as an engineering-style drawing.

CIRCUIT DIAGRAM



MACHINE SCHEMATIC

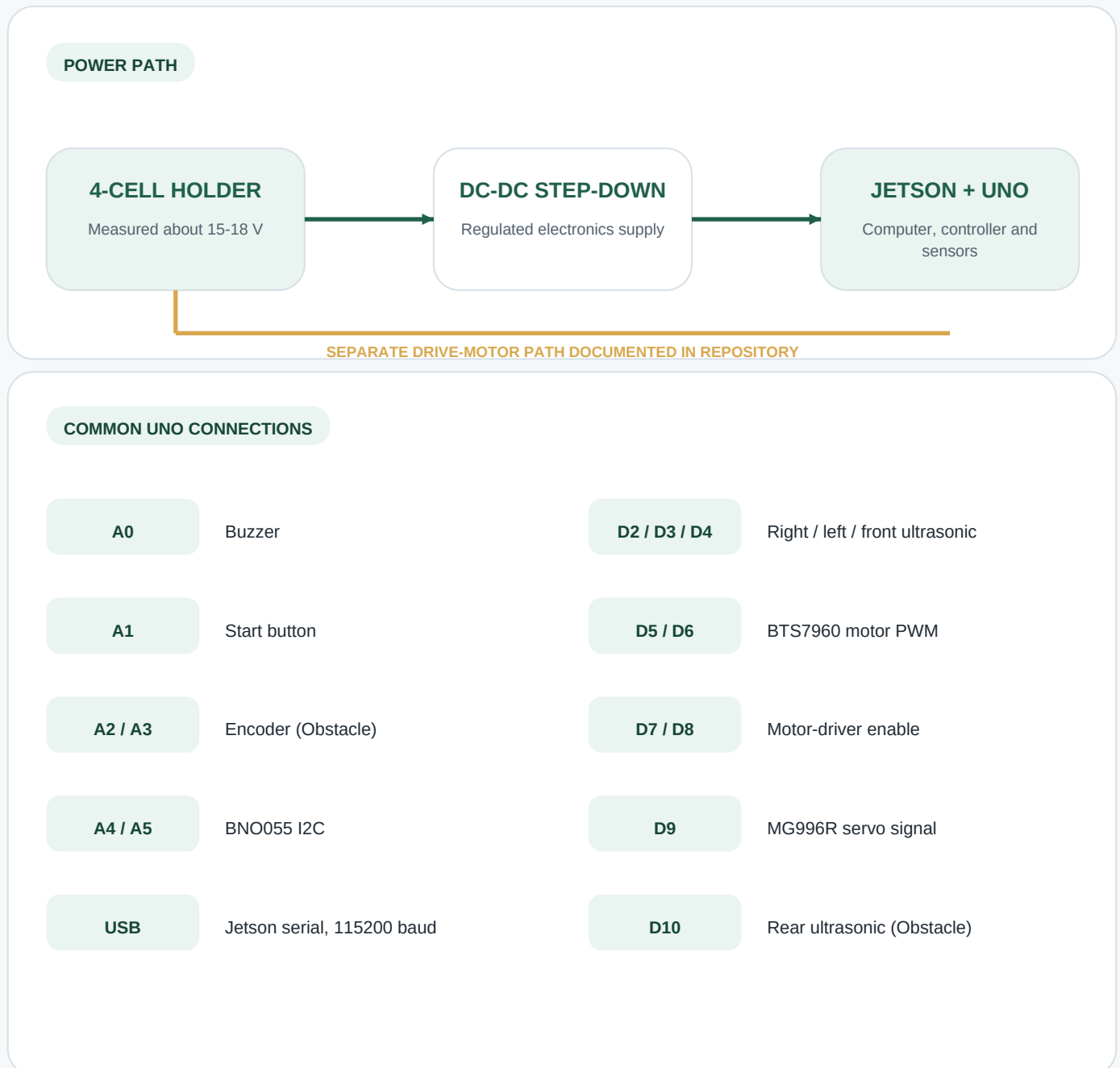


Use the hardware list and version-specific Uno pin map when checking the drawings.

Power & Pin Map

ELECTRICAL ORGANIZATION

Four 3.7-4.4 V, 5000 mAh cells provide approximately 15-18 V at the holder. A DC-DC step-down regulator supplies the electronics; the repository notes a separate direct motor path and identifies the motor as marked 7.2 V.



SAFETY NOTE

Disconnect drive power before wiring or calibration. Use adult/coach supervision, verify regulator output and keep a physical power disconnect available.

Software & Code

MATCHED PROGRAM PAIRS

The Jetson Python program and Arduino controller must match the selected challenge. Open uses three ultrasonic readings; Obstacle adds rear distance, encoder telemetry and distance-movement commands.

OPEN CHALLENGE

`open_challenge.py`

`open_challenge_controller.ino`

- Intersection-colour detection
- Straight-heading correction
- Corner turns and final stop

OBSTACLE CHALLENGE

`obstacle_challenge.py`

`obstacle_challenge_controller.ino`

- Pillar avoidance and parking
- Rear-distance telemetry
- Encoder-based distance moves

HIGH-LEVEL FLOW



Serial commands include START, STOP, CENTER and TURN_ABS. The Obstacle controller also supports FWD_CM and BACK_CM and reports rear distance plus encoder-centimetre telemetry.

TLM, yaw=..., dF=..., dL=..., dR=..., dB=..., enc_cm=...

Obstacle telemetry example; fields vary by controller.

IMPORTANT COMPATIBILITY

- The older `ird_max_controller.ino` is retained for development reference, not as the Obstacle controller.
- The replacement Uno R3 commonly appears as `/dev/ttyACM0`; the real listed port should always be checked.
- The uploaded controllers do not provide a general serial-loss watchdog for continuous driving.

Calibration & Setup

REPEATABLE PREPARATION

The setup guide keeps code versions, colour settings and controller firmware together. Calibration helps read telemetry and produce settings without replacing the robot's competition logic.

`autotune_colors.py`

Select blue, orange, red and green samples from the live camera and write HSV settings in the expected YAML format.

`sensor_check.py`

Display yaw, ultrasonic and encoder readings and optionally save them to CSV for comparison.

`encoder_calibrate.py`

Calculate a scale correction by rolling the robot by hand over a measured distance.

SETUP CHECKLIST

- Clone the repository and record the commit ID.
- Install the listed Python packages and verify that OpenCV receives camera frames.
- Upload the controller that matches the chosen challenge program.
- Find the real serial port, then verify READY, PING/PONG and sensor telemetry.
- Calibrate colours under the event lighting and repeat the encoder scale measurement.
- Test with raised wheels before a controlled floor run.

Testing & Results

EVIDENCE WITH CLEAR LIMITS

The testing folder records what each clip was intended to check and reports only what is visible. The procedure also defines what to record so later runs can be compared fairly.

01

Track driving & turning

The car crossed the track and began a corner. The clip ended early, so it is not presented as a completed run.

02

3D-printed body part

The part completed printing and its shape was checked before it was fitted to the vehicle.

03

Steering system

The servo moved the linkage and turned the front wheels during a table test.

REPEATABLE TEST RECORD

- Save date, Git commit, Python file, Uno firmware, colour YAML and track direction.
- Record lighting, wall/pillar layout, start position and the setting that changed.
- Use measurable outcomes: laps, time, interventions, contacts, heading error and distance error.
- Change one group of settings at a time and keep failed runs when they show the cause clearly.

[OPEN CHALLENGE VIDEO](#)
[OBSTACLE CHALLENGE VIDEO](#)

Problems & Solutions

DEVELOPMENT JOURNAL

The journal documents the problem, attempts, final change and observed result. This makes the engineering process visible instead of showing only the finished robot.

01

Faulty USB drive

- PROBLEM** JetPack installation would not finish.
- SOLUTION** Replace the USB drive and repeat the installation.
- RESULT** Installation completed with the replacement drive.

02

NoMachine connection

- PROBLEM** Laptop and Jetson used incompatible versions.
- SOLUTION** Install the same NoMachine version on both devices.
- RESULT** Remote access to the Jetson desktop worked.

03

Yaw and straight driving

- PROBLEM** The car steered left and right while it should have been straight.
- SOLUTION** Hold a target heading using fresh yaw readings, smooth corrections and reset correction history after turns.
- RESULT** The repository provides matched slow runs with and without the straight-heading system.

Problems & Solutions

DISTANCE MEASUREMENT

These entries record two distance-measurement faults discovered during full-field testing and the changes that restored repeatable movement.

04

Inconsistent distance

- PROBLEM** The programmed distance was inconsistent. The robot travelled a different distance after each full lap around the field, which includes four corner turns.
- SOLUTION** We created a separate distance-checker program and repeated the same movement several times. This allowed us to verify the distance values and adjust the calculation until the robot travelled a consistent and accurate distance.
- RESULT** The distance became more reliable and repeatable during full-field runs.

05

Worn encoder

- PROBLEM** The encoder had been used heavily during testing. Over time, its readings became lower and unreliable, even when the robot travelled the same distance.
- SOLUTION** We replaced the worn encoder with a new one and checked its readings using the distance-checker program.
- RESULT** The new encoder produced stable readings and restored accurate distance measurement.

Conclusion & Repository Guide

ENGINEERING SUMMARY

IRD MAX combines camera-based perception, heading and distance sensing, a two-board control system, front steering and rear drive. The repository connects the physical design to matched software, calibration, testing evidence and documented problem solving.



KEY STRENGTHS

- Current vehicle and boards are identified clearly.
- Open and Obstacle programs have matched Uno controllers.
- CAD, wiring and pin documentation describe the build from different viewpoints.
- Calibration and test procedures support repeatable work.
- The journal preserves failures, changes and observed results.

WHERE TO LOOK

README.md	Team, vehicle and repository overview
CAD/	Mechanical layout and six views
hardware/	Parts, power notes and Uno pin map
schematic/	Circuit and machine diagrams
software/	Challenge programs, protocol, setup and tools
testing/	Evidence entries and repeatable test procedure
Journal.md	Problems, attempts, solutions and outcomes

OPEN THE IRD MAX GITHUB REPOSITORY